

DEFENSIVE SECURITY TRAINING MODULE

HTTP Security:

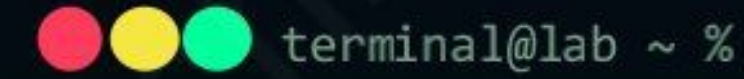
Understanding Filtering & Defense

How Attacks Work · How Defenses Stop Them · How to Harden Your Systems

FOR AUTHORIZED EDUCATIONAL USE ONLY

// ANALYSIS

- ▶ **Stateless protocol:** Each request is self-contained — no automatic session memory between calls
- ▶ **Request structure:** Method + URI + HTTP version + Headers + optional Body
- ▶ **Methods:** GET (retrieve), POST (submit data), PUT (replace), DELETE (remove resource)
- ▶ **Status codes:** 2xx success, 3xx redirect, 4xx client error, 5xx server error
- ▶ **Security implication:** Every part of the request is attacker-controlled unless cryptographically verified

terminal@lab ~ %

```
01 GET /products/123 HTTP/1.1
02 Host: shop.example.com
03 User-Agent: Mozilla/5.0
04 Accept: text/html
05 Connection: keep-alive
06
07 # Server response example:
08 HTTP/1.1 200 OK
09 Content-Type: text/html
10 Content-Length: 3821
11 X-Content-Type-Options: nosniff
12 X-Frame-Options: DENY
```



KEY INSIGHT

Every field in an HTTP request is supplied by the client and can be modified. Secure servers validate all input independently of the transport layer.

02 HTTP Headers – Structure & Security Role

// ANALYSIS

- ▶ **Host:** Identifies the virtual host — servers must validate this matches expected domains
- ▶ **Content-Type:** Declares body format — mismatches should be rejected, not coerced
- ▶ **Authorization:** Carries credentials — must be validated cryptographically on every request
- ▶ **Security headers:** Strict-Transport-Security, CSP, X-Frame-Options protect the client
- ▶ **Never trust:** Referer, X-Forwarded-For, User-Agent — all are fully client-controlled

terminal@lab ~ %

```
01 # Inspect response security headers
02 $ curl -sI https://yourapp.example.com
03
04 # Good security header set in response:
05 Strict-Transport-Security: max-age=31536000
06 Content-Security-Policy: default-src 'self'
07 X-Content-Type-Options: nosniff
08 X-Frame-Options: DENY
09 Referrer-Policy: no-referrer
10 Permissions-Policy: geolocation=()
11
12 # Check with securityheaders.com scanner
13 # or Mozilla Observatory for scoring
```



KEY INSIGHT

Security response headers instruct browsers to enforce protections. They are your last line of client-side defense — but server-side validation must never depend on them.

// ANALYSIS

- ▶ **IP allowlists:** Restrict access to known CIDR ranges — effective for internal APIs
- ▶ **WAF signature rules:** Pattern-match on URI, headers, and body for known attack strings
- ▶ **Rate limiting:** Throttle clients exceeding request thresholds to prevent abuse and DoS
- ▶ **Input validation:** Reject requests that do not conform to expected schema, type, and length
- ▶ **Defense-in-depth:** Filters are layered — WAF + app validation + DB parameterization

 terminal@lab ~ %

```
01 # Nginx rate limiting config (server-side)
02 limit_req_zone $binary_remote_addr
03     zone=api:10m rate=10r/s;
04
05 server {
06     location /api/ {
07         limit_req zone=api burst=20 nodelay;
08         limit_req_status 429;
09     }
10 }
11
12 # ModSecurity WAF rule example (detect)
13 SecRule REQUEST_URI "@contains /etc/passwd"
14     "id:1001,phase:1,deny,status:403,
15     msg:'Path traversal attempt detected'"

```

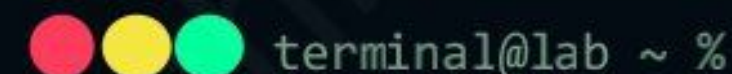


KEY INSIGHT

Filters catch known-bad patterns, but secure code catches unknown-bad input by only accepting what is explicitly valid (allowlist approach), not blocking what is known-bad (blocklist).

// ANALYSIS

- ▶ **400 Bad Request:** Reveals input validation rules — tune error messages to be non-descriptive
- ▶ **401 vs 403:** Distinguish 'not authenticated' from 'not authorized' — avoid leaking resource existence
- ▶ **500 Internal Server Error:** Stack traces in production expose framework, file paths, and logic
- ▶ **Timing differences:** Variable response times can reveal valid vs invalid usernames (user enumeration)
- ▶ **Defensive practice:** Return generic errors externally; log detailed errors internally only

terminal@lab ~ %

```
01 # Bad — Leaks internal detail to client:
02 HTTP/1.1 500 Internal Server Error
03 {
04     "error": "NullPointerException at
05     com.app.UserDAO.findById(UserDAO.java:47)",
06     "db_host": "postgres-prod-01",
07     "query": "SELECT * FROM users WHERE..."
08 }
09
10 # Good — generic external, detailed internal:
11 HTTP/1.1 500 Internal Server Error
12 { "error": "An unexpected error occurred",
13   "reference": "ERR-20240901-8821" }
14 # → Internal log has full stack trace
```

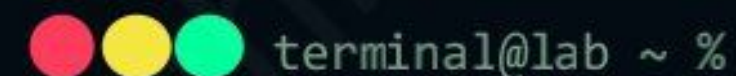


KEY INSIGHT

Error messages are a free information disclosure vulnerability. Never expose stack traces, database errors, file paths, or internal hostnames to external clients in production.

// ANALYSIS

- ▶ **Purpose:** Safely transmit special characters in URLs using %HH percent-encoding (RFC 3986)
- ▶ **Unreserved chars:** A-Z, 0-9, hyphen, dot, underscore, tilde — never need encoding
- ▶ **Reserved chars:** / ? # & = must be encoded when used as data, not structure: %2F %3F %23
- ▶ **Server decode order:** Servers decode before routing — validate after decoding, not before
- ▶ **Security rule:** Always decode fully before validating — attackers encode to bypass pre-decode checks

terminal@lab ~ %

```
01 # Python: safe URL encoding for params
02 import urllib.parse
03
04 # Encode user-supplied data before use in URL
05 user_input = "hello world & more"
06 safe = urllib.parse.quote(user_input)
07 print(safe) # hello%20world%20%26%20more
08
09 # Always decode before validating
10 raw = "%2Fetc%2Fpasswd"
11 decoded = urllib.parse.unquote(raw)
12 # decoded = "/etc/passwd" ← validate THIS
13
14 # Java equivalent
15 String decoded = URLDecoder.decode(raw, "UTF-8");
16 // Then validate decoded string
```



KEY INSIGHT

Validate input AFTER full decoding. If you validate encoded input, attackers encode their payload to bypass your checks — the server decodes it later and processes the dangerous value.

06 Encoding Attacks & How to Prevent Them

// ANALYSIS

- ▶ **Double encoding:** %252F encodes the % sign — server decodes twice and gets / after validation
- ▶ **Why it works:** Filter decodes once and sees %2F (harmless), backend decodes again to / (dangerous)
- ▶ **Prevention:** Normalize and fully decode all input exactly once before any validation logic runs
- ▶ **Path traversal via encoding:** ../%2F and %2E%2E%2F both mean ../ after decoding
- ▶ **Remediation:** Use canonical path resolution (realpath/Path.normalize) after decoding, before access

```
terminal@lab ~ %  
  
01 # Java: safe file access with canonicalization  
02 import java.io.File;  
03  
04 String userInput = request.getParameter("file");  
05 File base = new File("/var/app/uploads/");  
06 File requested = new File(base, userInput);  
07  
08 // Canonical path resolves ALL encodings + ../  
09 String canonical = requested.getCanonicalPath();  
10 String basePath = base.getCanonicalPath();  
11  
12 if (!canonical.startsWith(basePath)) {  
13     // Path traversal attempt – reject  
14     throw new SecurityException("Invalid path");  
15 }  
16 // Only reach here if path is safe  
17 return Files.readAllBytes(requested.toPath());
```



KEY INSIGHT

Canonicalization is the correct defense against path traversal and encoding attacks. Resolve the real path after decoding and verify it starts with your allowed base directory before any file access.

// ANALYSIS

- ▶ **Host header injection:** Unsanitized Host used in password-reset links leads to account takeover
- ▶ **X-Forwarded-For abuse:** Apps trusting this for IP-based access control can be trivially bypassed
- ▶ **CORS misconfiguration:** Reflecting Origin header back without validation allows cross-site data theft
- ▶ **Content-Type confusion:** Accepting wrong content types may bypass input validation middleware
- ▶ **Defense:** Validate Host against an explicit allowlist; never use client headers for trust decisions

terminal@lab ~ %

```
01 # Secure Host header validation (Python/Flask)
02 from flask import request, abort
03
04 ALLOWED_HOSTS = {"app.example.com", "www.example.com"}
05
06 @app.before_request
07 def validate_host():
08     host = request.host.split(":")[0] # strip port
09     if host not in ALLOWED_HOSTS:
10         abort(400) # reject unknown Host values
11
12 # Secure CORS – explicit origin allowlist
13 ALLOWED_ORIGINS = {"https://app.example.com"}
14
15 @app.after_request
16 def set_cors(response):
17     origin = request.headers.get("Origin", "")
18     if origin in ALLOWED_ORIGINS:
19         response.headers["Access-Control-Allow-Origin"]
20         = origin
21     return response
```



KEY INSIGHT

Never reflect client-supplied headers back without validation. A strict allowlist of known-good values — for Host, Origin, and Content-Type — eliminates entire classes of injection vulnerability.

// ANALYSIS

- ▶ curl is a standard tool for testing your own application's HTTP behavior and header responses
- ▶ **-I / --head**: Fetch only response headers — check security header configuration quickly
- ▶ **-v / --verbose**: Show full request + response for debugging auth and filter behavior
- ▶ **--proxy**: Route through Burp Suite to inspect and replay requests on your own test environment
- ▶ **Scope rule**: Only use against systems you own or have explicit written authorization to test

```
terminal@lab ~ %  
  
01 # Check your app's security headers  
02 $ curl -sI https://yourapp.example.com \  
03     | grep -iE "strict|csp|x-frame|referrer"  
04  
05 # Test your own API endpoint response  
06 $ curl -v -X POST https://yourapp.example.com/api/login \  
07     -H "Content-Type: application/json" \  
08     -d '{"user":"testuser","pass":"testpass"}'  
09  
10 # Verify rate limiting is working  
11 $ for i in {1..15}; do  
12     curl -so /dev/null -w "%{http_code}\n" \  
13     https://yourapp.example.com/api/login  
14 done  
15 # Should see 429 after threshold
```



KEY INSIGHT

curl is invaluable for validating your own defenses — check headers, confirm rate limiting, verify authentication flows. Always test against environments you control or are authorized to assess.

// ANALYSIS

- ▶ nc sends raw bytes over TCP — useful for testing how your server handles malformed requests
- ▶ **Use case:** Verify your server returns 400 on malformed input rather than 500 (implementation error)
- ▶ **Authorized use:** Test your own server's parser robustness and error handling behavior
- ▶ **ncat (nmap):** Supports TLS via --ssl for testing HTTPS endpoints you own
- ▶ **Lab only:** Raw TCP testing is for controlled lab environments — never against production or third-party

```
terminal@lab ~ %  
  
01 # Test your own server's error handling  
02 # Send a malformed request – expect 400  
03 $ printf "BADMETHOD / HTTP/1.1\r\n\  
04 Host: localhost\r\n\  
05 \r\n" | nc localhost 8080  
06  
07 # Expected secure response:  
08 HTTP/1.1 400 Bad Request  
09 Content-Type: text/plain  
10 Connection: close  
11  
12 # Verify your server closes connection  
13 # cleanly on malformed input (not 500)  
14  
15 # Test TLS config on your own server  
16 $ ncat --ssl localhost 443
```



KEY INSIGHT

A hardened server returns 400 (not 500) for malformed requests and closes the connection cleanly. Testing error handling on your own server helps you confirm the implementation is robust, not just the happy path.

// ANALYSIS

- ▶ **What it is:** Using ../ sequences in file paths to navigate outside the intended directory
- ▶ **Common target:** File download/upload endpoints, template engines, log viewers accepting filenames
- ▶ **Why encoding helps attackers:** %2F, %252F, and ../%2F all resolve to / after server decoding
- ▶ **Prevention rule 1:** Canonicalize the full path and verify it begins with your allowed base path
- ▶ **Prevention rule 2:** Never construct file paths from user input — use indirect references (ID → path map)

```

terminal@lab ~ %
01 # Secure indirect file reference (Python)
02 import os
03
04 # Map IDs to real files – user never touches path
05 FILE_MAP = {
06     "report_q1": "/var/reports/q1_2024.pdf",
07     "report_q2": "/var/reports/q2_2024.pdf",
08 }
09
10 def get_file(file_id):
11     # Reject anything not in the allowlist
12     path = FILE_MAP.get(file_id)
13     if path is None:
14         raise ValueError("Invalid file reference")
15     # Double-check canonical path
16     real = os.path.realpath(path)
17     allowed = os.path.realpath("/var/reports/")
18     if not real.startswith(allowed):
19         raise ValueError("Path traversal detected")
20     return open(path, "rb").read()

```



KEY INSIGHT

Indirect references are the strongest defense — users reference an ID, never a file path. If you must use user-supplied paths, canonicalize and verify against your base directory before any file operation.

// ANALYSIS

- ▶ **Allowlist approach:** Define exactly what valid input looks like — reject everything else
- ▶ **Validate after decoding:** Decode URL encoding, HTML entities, and Unicode normalization first
- ▶ **Type + length + format:** Check data type, enforce max length, and validate format (regex/schema)
- ▶ **Reject early:** Validate at the boundary (controller/API layer) before the value enters business logic
- ▶ **Output encoding:** Encode data for its output context (HTML, SQL, JSON) separately from input validation

terminal@lab ~ %

```

01 # Secure input validation in Python
02 import re
03 from html import escape
04
05 def validate_username(raw: str) -> str:
06     # 1. Decode (framework usually handles this)
07     # 2. Type check
08     if not isinstance(raw, str):
09         raise ValueError("Username must be a string")
10     # 3. Length check
11     if not (3 <= len(raw) <= 32):
12         raise ValueError("Username length invalid")
13     # 4. Format allowlist (alphanumeric + underscore)
14     if not re.fullmatch(r"[a-zA-Z0-9_]+", raw):
15         raise ValueError("Username contains invalid
chars")
16     return raw # validated, safe to use
17
18 # Always use parameterized queries – no string concat
19 cursor.execute("SELECT * FROM users WHERE name = %s",
20                (validated_username,))

```

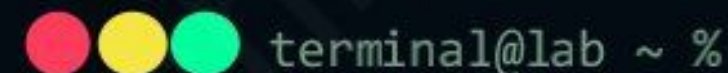


KEY INSIGHT

Allowlist validation is the single most effective input security control. Define what is valid — not what is dangerous. Parameterized queries eliminate SQL injection independently of input validation.

// ANALYSIS

- ▶ WAFs are a compensating control — they add a filtering layer but do not fix insecure code
- ▶ **Signature rules:** Match known attack patterns (SQLi, XSS, traversal) in URI, headers, and body
- ▶ **False positive risk:** Overly aggressive rules block legitimate traffic — tune carefully with monitoring
- ▶ **Bypass reality:** Obfuscated or novel payloads may evade WAF rules — defense-in-depth is essential
- ▶ **What a WAF cannot do:** Fix SQL injection in code, patch server vulnerabilities, or replace auth logic

terminal@lab ~ %

```
01 # ModSecurity + OWASP CRS basic setup (Apache)
02 # /etc/modsecurity/modsecurity.conf
03 SecRuleEngine On
04 SecRequestBodyAccess On
05 SecResponseBodyAccess On
06 SecResponseBodyMimeType text/plain text/html
07
08 # Load OWASP Core Rule Set
09 Include /etc/modsecurity/crs/crs-setup.conf
10 Include /etc/modsecurity/crs/rules/*.conf
11
12 # Log blocked requests for tuning
13 SecAuditEngine RelevantOnly
14 SecAuditLogParts ABIJDEFHZ
15 SecAuditLog /var/log/modsec_audit.log
16
17 # Monitor, don't just block - tune first
18 SecDefaultAction "phase:1,log,pass"
```



KEY INSIGHT

A WAF in Detection mode (logging only) lets you tune rules for 2-4 weeks before enabling blocking — preventing false positives. Start with OWASP CRS at paranoia level 1, then increase after tuning.

// ANALYSIS

- ▶ **Written authorization:** Obtain explicit written scope agreement before any testing activity
- ▶ **Scope definition:** Agree on target systems, IP ranges, test window, and out-of-scope systems
- ▶ **Recon phase:** Map endpoints and responses using only authorized, non-destructive techniques
- ▶ **Testing phase:** Probe for vulnerabilities methodically — document every finding with evidence
- ▶ **Reporting:** Deliver findings with severity ratings, reproduction steps, and remediation guidance

 terminal@lab ~ %

```
01 # Example: authorized endpoint enumeration
02 # Only against systems in your written scope
03
04 $ cat scope.txt
05 # Authorized targets (from signed agreement):
06 # 192.168.10.0/24 - staging environment
07 # dev.example.com - development server
08
09 # Passive header check - non-destructive
10 $ curl -sI https://dev.example.com \
11   | tee findings/headers_$(date +%F).txt
12
13 # Document all findings immediately
14 $ cat > findings/finding_001.md << EOF
15 Title: Missing HSTS header
16 Severity: Medium
17 Evidence: curl -sI https://dev.example.com
18 Remediation: Add Strict Transport Security header
19 EOF
```



KEY INSIGHT

Penetration testing without written authorization is illegal regardless of intent or technique. Authorized testing is a professional service — scope, document, and report every finding responsibly.

// ANALYSIS

- ▶ **Headers:** Deploy HSTS, CSP, X-Frame-Options, X-Content-Type-Options on all responses
- ▶ **Input:** Validate and sanitize all input server-side; use parameterized queries everywhere
- ▶ **Auth:** Enforce HTTPS only, use short-lived tokens, implement MFA for privileged access
- ▶ **Errors:** Return generic errors externally; log detailed diagnostics internally with correlation IDs
- ▶ **Monitor:** Centralize logs, alert on anomalous patterns, review WAF blocks weekly

terminal@lab ~ %

```
01 # NGINX hardened header config
02 add_header Strict-Transport-Security
03     "max-age=31536000; includeSubDomains" always;
04 add_header Content-Security-Policy
05     "default-src 'self'; script-src 'self'" always;
06 add_header X-Frame-Options "DENY" always;
07 add_header X-Content-Type-Options "nosniff" always;
08 add_header Referrer-Policy "no-referrer" always;
09
10 # Hide server version
11 server_tokens off;
12
13 # Limit accepted HTTP methods
14 if ($request_method !~ ^(GET|POST|PUT|DELETE)$) {
15     return 405;
16 }
17
18 # Body size limit (prevent large payload abuse)
19 client_max_body_size 1m;
```



KEY INSIGHT

Security hardening is not one task — it is an ongoing posture. Automate header checks in your CI/CD pipeline, scan monthly with tools like Mozilla Observatory, and patch dependencies on a regular schedule.

KEY TAKEAWAYS

- 01 Validate and sanitize all input server-side — never trust headers, URLs, or body data from clients
- 02 WAF rules complement secure code — they are not a substitute for input validation and output encoding
- 03 HTTP headers like X-Forwarded-For and Referer are client-controlled and must never drive access decisions
- 04 URL decode and normalize all input before validation — process it the same way the backend will
- 05 Test your own systems with authorized tools; document findings and remediate before attackers find them